

Relatório de Implementação – Lab 4

Projeto: All Invest · **Componentes:** Investment Component e Portfolio Component

Integrantes:

- | | |
|---|---|
| 1. Danilo Henrique de Paulo 22.222.008-9 | 4. Pedro Henrique da Fonseca do Nascimento 22.123.099-8 |
| 2. Jônatas da Silva Gonçalves 22.123.095-6 | 5. Vinícius do Nascimento Generoso 22.123.103-8 |
| 3. Kaynã de Deus Ferreira da Silva 22.123.096-4 | 6. Wallace dos Santos Izidoro 22.123.104-6 |

1. Resumo

Foram implementados os dois componentes definidos no Lab 3: o **Portfolio Component**, que não existia, e o **Investment Component**, que já existia e passou a **requerer a interface** `IPortfolioService`. Com isso, o usuário pode criar e gerenciar carteiras, registrar operações já associadas a uma carteira, mover ativos entre carteiras e consultar as posições por carteira.

Resultado dos testes: **45 passed, 4 warnings in 1.96s** (45 testes; execução registrada em docs/evidencias/).

2. Ponto de partida

- O backend (FastAPI + SQLAlchemy) já organizava os serviços como componentes, com interfaces Protocol em `interfaces/components.py` e injeção de dependências em `dependencies/components.py`.
- O **Investment Component** já registrava compras e vendas e calculava o preço médio, mas não tinha relação com carteiras.
- O **Portfolio Component** do Lab 3 não existia: não havia tabelas, serviço nem rotas de carteira.
- Não havia testes automatizados no projeto.

3. O que foi feito

3.1 Arquivos criados

Arquivo	Conteúdo
backend/src/services/portfolio_service.py	Portfolio Component: implementação de <code>IPortfolioService</code> (carteiras e associação ativo → carteira).
backend/src/routers/portfolios.py	Rotas REST <code>/api/portfolios</code> (criar, listar, consultar, editar, remover).
database/migrations/002_create_portfolios.sql	Cria as tabelas <code>portfolios</code> e <code>portfolios_stocks</code> (migração só aditiva).
backend/tests/ (conftest e 5 arquivos)	45 testes: unitários dos dois componentes, integração entre eles, ponta a ponta pela API e conformidade com as interfaces.
backend/pytest.ini, backend/requirements-dev.txt	Configuração do pytest e dependências de desenvolvimento.
.github/workflows/tests.yml	Executa os testes no GitHub Actions a cada push e pull request.

Arquivo	Conteúdo
docs/evidencias/resultado_testes.txt e .xml	Saída da execução dos testes (texto e JUnit XML).
Especificacao_Componentes.pdf, Roteiro_Video.pdf, Relatorio_Implementacao.pdf	Documentos da entrega, na raiz do projeto.

3.2 Arquivos alterados

Arquivo	Alteração
backend/src/interfaces/components.py e __init__.py	Nova IPortfolioService; IInvestmentService ganhou associate_position e o filtro por carteira. Contratos documentados nas docstrings.
backend/src/services/investment_service.py	Passa a requerer IPortfolioService no construtor; create_trade aceita portfolio_id; novo associate_position; get_positions informa e filtra a carteira; remove_position desfaz a associação.
backend/src/dependencies/components.py	Novo get_portfolio_service; o Portfolio é injetado no Investment pela interface.
backend/src/models.py	Modelos Portfolio e PortfolioStock (FK composta garante que a carteira é do mesmo usuário).
backend/src/db/schemas.py	DTOs PortfolioCreate, PortfolioUpdate, PortfolioOut e PortfolioAssignment; portfolio_id opcional em StockTradeCreate; portfolio_id e portfolio_name em PositionOut.
backend/src/db/crud.py	Funções de persistência das carteiras e das associações.
backend/src/routers/stocks.py	Filtro ?portfolio_id= em /api/stocks/carteira e nova rota PUT /api/stocks/{ticker}/portfolio.
backend/src/main.py	Registra o router de carteiras.
database/CreateTables.sql	Inclui as novas tabelas para instalações do zero.
README.md	Seções exigidas: tecnologias, estrutura dos componentes, interfaces e operações, execução, testes e comunicação entre os componentes.

O frontend não foi alterado. As respostas ganharam apenas campos opcionais, então a tela atual continua funcionando.

4. Como funciona

4.1 Arquitetura

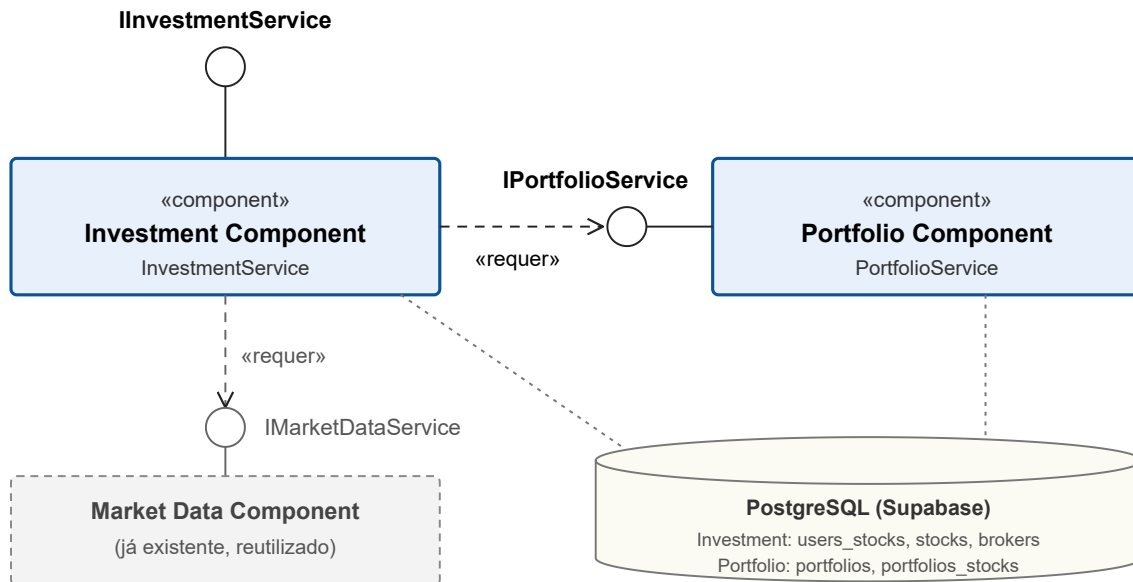


Figura 1 – Componentes implementados, interfaces e dados.

4.2 Modelo de dados do Portfolio

Tabela	Colunas	Regras
portfolios	portfolio_id, user_id, portfolio_name, description, created_at	Nome único por usuário. Removida em cascata se o usuário for removido.
portfolios_stocks	user_id, stock_id, portfolio_id	Chave (user_id, stock_id): cada ativo do usuário fica em no máximo uma carteira. A FK composta (portfolio_id, user_id) impede usar carteira de outro usuário.

4.3 Comunicação entre os componentes

O Investment conhece apenas a interface. O FastAPI monta a cadeia em cada requisição: cria o PortfolioService e o entrega ao InvestmentService, com a mesma sessão de banco. A Figura 2 mostra o registro de uma compra já associada a uma carteira.

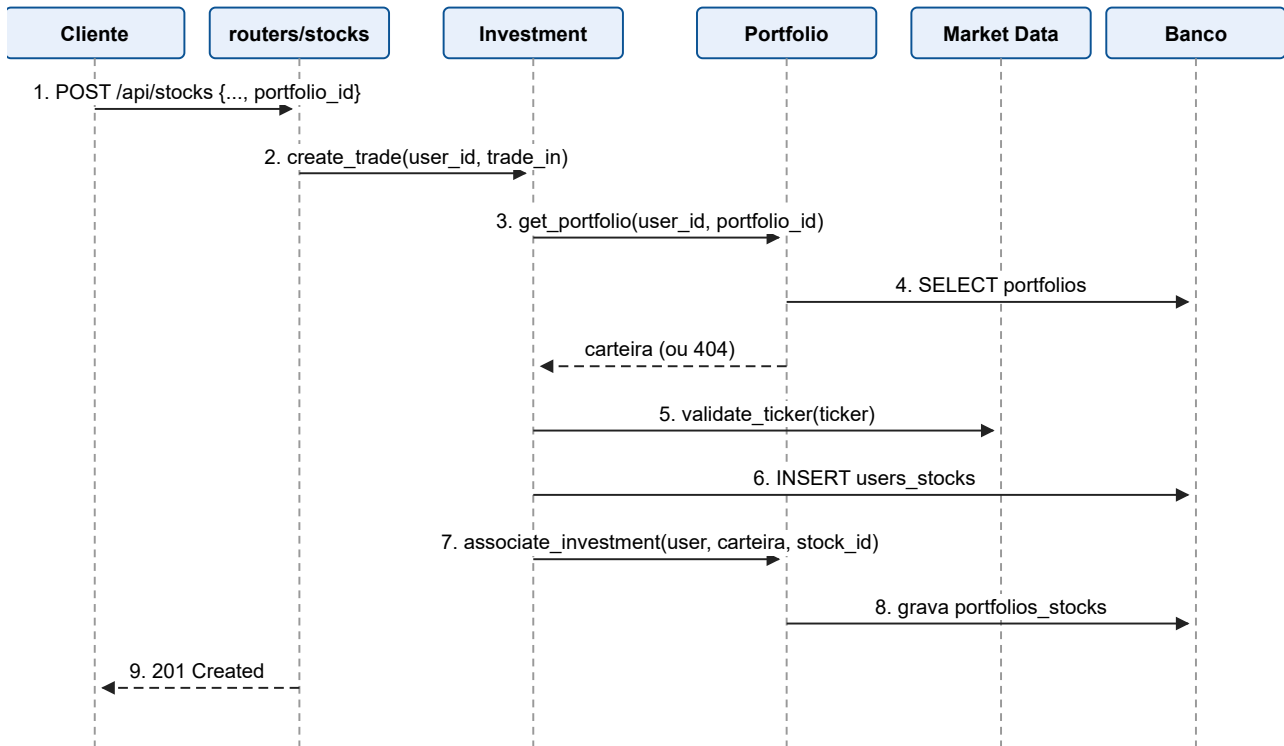


Figura 2 – Sequência de POST /api/stocks com portfolio_id.

A carteira é validada **antes** de consultar a B3 e gravar a operação. Por isso, uma carteira inválida devolve 404 sem gravar nada, como verifica o teste

test_create_trade_com_carteira_inexistente_nao_grava_operacao.

4.4 Endpoints novos ou alterados

Método e rota	Componente / operação
POST /api/portfolios	Portfolio · create_portfolio
GET /api/portfolios	Portfolio · list_portfolios
GET /api/portfolios/{id}	Portfolio · get_portfolio
PATCH /api/portfolios/{id}	Portfolio · update_portfolio
DELETE /api/portfolios/{id}	Portfolio · delete_portfolio
POST /api/stocks (campo portfolio_id opcional)	Investment · create_trade → Portfolio
GET /api/stocks/carteira?portfolio_id=	Investment · get_positions → Portfolio
PUT /api/stocks/{ticker}/portfolio	Investment · associate_position → Portfolio
DELETE /api/stocks/{ticker}	Investment · remove_position → Portfolio

5. Testes e evidências

Os testes usam SQLite em memória e um stub do Market Data. Não acessam o Supabase nem a brapi.dev, e rodam com `python -m pytest -v` dentro de `backend/`.

Arquivo	Tipo	Testes
<code>test_portfolio_component.py</code>	Unitário – Portfolio	14
<code>test_investment_component.py</code>	Unitário – Investment	15
<code>test_integration_investment_portfolio.py</code>	Integração – componentes	7
<code>test_api_integration.py</code>	Integração – API HTTP	5
<code>test_interface_contracts.py</code>	Contrato das interfaces	4
Total		45

Execução registrada: 2026-10-03 16:55:46 — comando: `python -m pytest -v` (pasta `backend/`). Resultado: **45 passed, 4 warnings in 1.96s**. Os avisos são de depreciação de bibliotecas de terceiros e não afetam os testes.

5.1 Lista dos testes executados

Tipo	Teste	Resultado
Integração – API HTTP	<code>test_fluxo_completo_carteira_e_investimento</code>	PASSED
Integração – API HTTP	<code>test_registrar_operacao_em_carteira_inexistente_retorna_404</code>	PASSED
Integração – API HTTP	<code>test_criar_carteira_com_nome_repetido_retorna_409</code>	PASSED
Integração – API HTTP	<code>test_editar_carteira_sem_campos_retorna_422</code>	PASSED
Integração – API HTTP	<code>test_rotas_de_carteira_exigem_api_key</code>	PASSED
Integração – componentes	<code>test_registrar_operacao_com_carteira_associa_ativo_no_portfolio</code>	PASSED
Integração – componentes	<code>test_associar_posicao_existente_e_mover_entre_carteiras</code>	PASSED
Integração – componentes	<code>test_consultar_posicoes_por_carteira</code>	PASSED
Integração – componentes	<code>test_remover_carteira_preserva_operacoes_e_desassocia_posicoes</code>	PASSED
Integração – componentes	<code>test_remover_investimento_remove_associacao_no_portfolio</code>	PASSED
Integração – componentes	<code>test_ nao Associa ativo a carteira de outro usuario</code>	PASSED
Integração – componentes	<code>test_filtrar_por_carteira_inexistente_retorna_404</code>	PASSED
Contrato das interfaces	<code>test_implementacao_fornece_todas_as_operacoes_da_interface[Portfolio]</code>	PASSED
Contrato das interfaces	<code>test_implementacao_fornece_todas_as_operacoes_da_interface[Investment]</code>	PASSED
Contrato das interfaces	<code>test_interfaces_expoem_as_operacoes_do_lab3</code>	PASSED
Contrato das interfaces	<code>test_investment_requer_portfolio_pelo_construtor</code>	PASSED



Tipo	Teste	Resultado
Unitário – Investment	test_create_trade_valida_ticker_e_persiste_compra	PASSED
Unitário – Investment	test_create_trade_com_auto_cotacao_usa_preco_do_market_data	PASSED
Unitário – Investment	test_create_trade_ticker_inexistente_retorna_404	PASSED
Unitário – Investment	test_create_trade_corretora_inexistente_retorna_400	PASSED
Unitário – Investment	test_create_trade_venda_acima_do_saldo_retorna_422	PASSED
Unitário – Investment	test_create_trade_com_carteira_valida_e_associa	PASSED
Unitário – Investment	test_create_trade_com_carteira_inexistente_nao_grava_operacao	PASSED
Unitário – Investment	test_get_positions_calcula_quantidade_e_preco_medio	PASSED
Unitário – Investment	test_get_positions_oculta_ativo_zerado	PASSED
Unitário – Investment	test_get_positions_isola_usuarios	PASSED
Unitário – Investment	test_get_positions_filtrar_por_carteira	PASSED
Unitário – Investment	test_associate_position_retorna_posicao_com_carteira	PASSED
Unitário – Investment	test_associate_position_sem_posicao_retorna_404	PASSED
Unitário – Investment	test_remove_position_apaga_operacoes_e_desassocia	PASSED
Unitário – Investment	test_remove_position_inexistente_retorna_404	PASSED
Unitário – Portfolio	test_create_portfolio_persiste_carteira_do_usuario	PASSED
Unitário – Portfolio	test_create_portfolio_rejeita_nome_repetido_sem_diferenciar_maiusculas	PASSED
Unitário – Portfolio	test_create_portfolio_permite_mesmo_nome_para_usuarios_diferentes	PASSED
Unitário – Portfolio	test_portfolio_create_rejeita_nome_vazio	PASSED
Unitário – Portfolio	test_list_portfolios_retorna_somente_as_do_usuario_ordenadas	PASSED
Unitário – Portfolio	test_get_portfolio_de_outro_usuario_retorna_404	PASSED
Unitário – Portfolio	test_update_portfolio_altera_somente_campos_enviados	PASSED
Unitário – Portfolio	test_update_portfolio_rejeita_nome_de_outra_carteira	PASSED
Unitário – Portfolio	test_portfolio_update_exige_ao_menos_um_campo	PASSED
Unitário – Portfolio	test_delete_portfolio_remove_carteira_e_associacoes	PASSED
Unitário – Portfolio	test_delete_portfolio_de_outro_usuario_retorna_404	PASSED
Unitário – Portfolio	test_associate_investment_associa_e_move_entre_carteiras	PASSED
Unitário – Portfolio	test_associate_investment_em_carteira_de_outro_usuario_retorna_404	PASSED
Unitário – Portfolio	test_dissociate_investment_informa_se_havia_associacao	PASSED

6. Como executar

```
# Testes
cd backend
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
python -m pytest -v

# API (após executar a migração 002 no Supabase)
uvicorn src.main:app --reload --port 8000
# Swagger: http://localhost:8000/docs (grupos "portfolios" e "stocks")
```

7. Mapa dos entregáveis

Item do enunciado	Onde está
1. Documento da especificação dos componentes	Especificacao_Componentes.pdf (template FEI preenchido, com contratos e alterações justificadas).
2. Implementação: código dos dois componentes	portfolio_service.py e investment_service.py.
2. Implementação: interfaces e operações	interfaces/components.py e rotas em routers/.
2. Implementação: integração	dependencies/components.py (Investment recebe IPortfolioService).
3. Testes das operações	test_portfolio_component.py, test_investment_component.py.
3. Testes da interação	test_integration_investment_portfolio.py, test_api_integration.py.
3. Evidências	docs/evidencias/ e a seção 5 deste relatório.
4. Repositório GitHub (código, testes, README)	Pronto para versionar; ver "Próximos passos".
5. Vídeo de 5 a 8 min	Roteiro_Video.pdf (cerca de 7min30s, cobre os itens 1 a 6).

8. Próximos passos para a equipe

- Executar a migração no Supabase** (database/migrations/002_create_portfolios.sql, no SQL Editor) antes de usar /api/portfolios no banco real. A migração ainda não foi aplicada, pois os testes rodam em SQLite em memória e não acessam o Supabase.
- Versionar e enviar ao GitHub.** Esta pasta ainda não é um repositório git. O .gitignore já ignora .env e .venv; confirmem com git status que nenhum .env aparece antes do primeiro commit.
- Gravar o vídeo** seguindo o Roteiro_Video.pdf. A demonstração usa o Swagger (/docs), pois o frontend ainda não tem tela de carteiras.
- Opcional:** criar no frontend a tela de carteiras e o seletor de carteira no formulário de operação. A API já está pronta e é compatível com a tela atual.